ShipFast — Provision Infrastructure with Terraform

Week 4, Session 2 assignment · Capstone Milestone 3 · Mohammed Awwad · 28 October 2026

REPOSITORY	BUCKET	REGION	ACCOUNT	TERRAFORM	PROVIDER	FIRST APPLY
github.com/ MohammedAwwad2/ shipfast-devops	created and live (see Part D)	us-east-1	IAM user, not root	v1.17.0 on linux_amd64 (WSL2)	hashicorp/aws 5.100.0	28 Oct 2026, 01:22

Every command in this report was run on 28 October 2026 and its output captured verbatim into `docs/evidence/m3/`. Nothing here is illustrative. No access key appears in the code, in the state, or in this document, and the account ID is masked throughout.

Part A — Installing Terraform

Terraform was installed inside WSL2 (Ubuntu) by following the official HashiCorp documentation rather than by downloading a zip by hand: add the HashiCorp GPG key to `/usr/share/keyrings/`, register the apt repository signed by that key, then `apt install terraform`. That way `apt upgrade` keeps it current and the binary's provenance is verifiable.

```
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor \
     -o /usr/share/keyrings/hashicorp-archive-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
     https://apt.releases.hashicorp.com $(lsb_release -cs) main" \
     | sudo tee /etc/apt/sources.list.d/hashicorp.list

sudo apt update && sudo apt install terraform -y
```

Both files landed at 00:58 on 28 October, and the package manager now owns the binary. The pre-flight checks from the Week 4 progress check pass:

```
$ cat /etc/apt/sources.list.d/hashicorp.list
deb [arch=amd64 signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
    https://apt.releases.hashicorp.com resolute main

$ dpkg -l terraform
ii  terraform   1.17.0-1   amd64   Terraform enables you to safely and predictably create, change ...

$ terraform version
Terraform v1.17.0
on linux_amd64

$ aws sts get-caller-identity
{
    "UserId": "AIDAXA74...",
    "Account": "4831*******",
    "Arn": "arn:aws:iam::4831*******:user/adminUser"
}
```

Terraform 1.17.0 is above the 1.6.0 floor the assignment requires, and the caller is an IAM user, not the root account. Evidence: `docs/evidence/m3/00-install-and-identity.txt`.

Part B — The folder, and keeping state out of Git

The `terraform/` folder was created inside the capstone repo at 01:01. The `.gitignore` already written in Week 3 covers Terraform: the provider cache and every state file are ignored, while the provider lock file is not. That split is deliberate — the lock file belongs in Git because it pins the provider version for everyone on the team, and the state file must not, because it is a plaintext dump of every resource attribute.

```
# .gitignore (the Terraform part)
.terraform/
*.tfstate*

$ git ls-files | grep -E "tfstate|\.terraform/"      # prints nothing
$ git ls-files terraform/
terraform/.terraform.lock.hcl
terraform/main.tf
```

Part C — Writing main.tf

All nine TODOs in the starter template were filled in. Three of them carried the decisions worth writing down:

The provider carries only a region.

No access keys, anywhere. Terraform reads the same credential chain the AWS CLI does, so the `aws configure` profile from A1 is already enough. A key hard-coded here would end up in Git and in state.

The bucket name is computed, never typed.

S3 bucket names are global across every AWS account, so a literal name would collide with a stranger's bucket. A `data "aws_caller_identity"` lookup asks AWS who the caller is, and the account ID in the name makes it unique without hard-coding anything.

Versioning and encryption are separate resources.

In AWS provider 5.x these are no longer inline blocks on the bucket; they are their own resources that point back at it by `aws_s3_bucket..id`. That reference is also what gives Terraform the ordering — the bucket has to exist before anything can be configured on it.

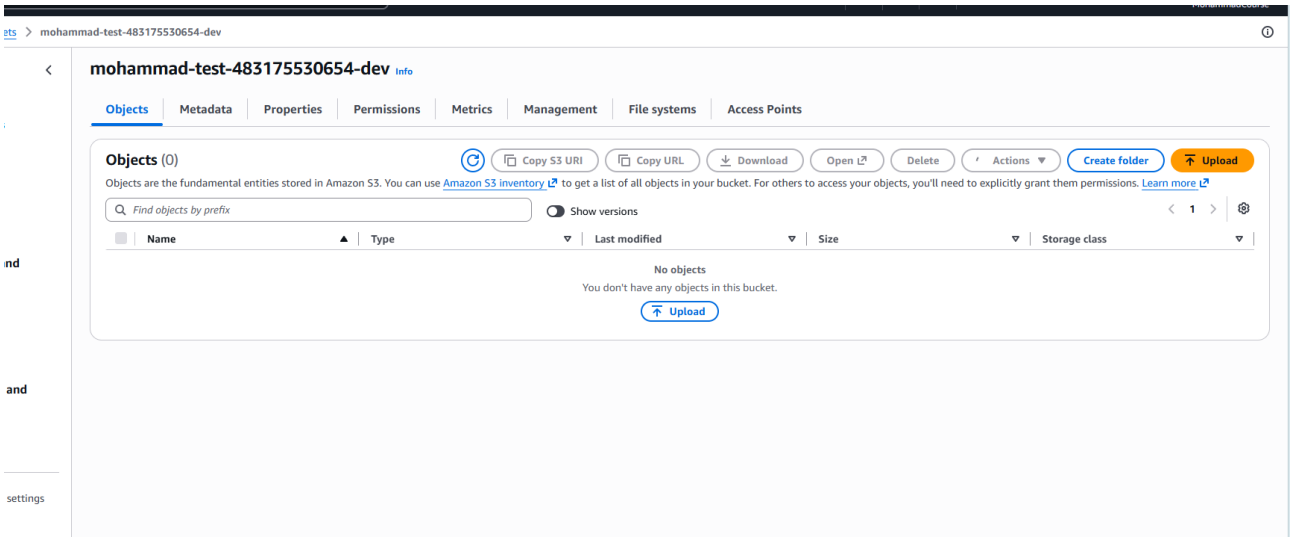
Part D — init and apply

`terraform init` ran at 01:18. It downloaded hashicorp/aws 5.100.0 into `.terraform/` and wrote `.terraform.lock.hcl` a few seconds later. `terraform apply` followed at 01:22 and created all three resources — the bucket, its versioning configuration and its default encryption. The state file records the apply, and the bucket is live in the account.

```
$ cd terraform
$ terraform init      # Installing hashicorp/aws v5.100.0 ... Terraform has been successfully initialized!
$ terraform apply     # 3 added, 0 changed, 0 destroyed
```

Committed as `feat: add terraform infrastructure for shipfast`.

The bucket in the S3 console



S3 console, Objects tab, straight after the first apply — the bucket exists and is empty, which is what the assignment expects at this stage. Blur the account ID before sharing this outside the repo.

Part E — Format, validate, verify

`terraform fmt` rewrote `main.tf` into canonical form (it printed the file name, which is how fmt reports a file it changed), and `terraform validate` passed. Verification of the result then deliberately used the AWS CLI rather than Terraform — asking Terraform whether Terraform worked only proves the state file agrees with itself. The CLI asks AWS directly, and both controls the assignment asks for came back on:

```
$ terraform fmt
main.tf

$ terraform validate
Success! The configuration is valid.

$ terraform output
bucket_arn  = "arn:aws:s3:::shipfast-artifacts-dev-4831*******"
bucket_name = "shipfast-artifacts-dev-4831*******"

$ aws s3api get-bucket-versioning --bucket shipfast-artifacts-dev-4831*******
{
    "Status": "Enabled"
}

$ aws s3api get-bucket-encryption --bucket shipfast-artifacts-dev-4831*******
{
    "ServerSideEncryptionConfiguration": {
        "Rules": [
            {
                "ApplyServerSideEncryptionByDefault": { "SSEAlgorithm": "AES256" },
                "BucketKeyEnabled": false
            }
        ]
    }
}
```

Versioning `Enabled`, default encryption `AES256`, and both outputs resolve. Evidence: `docs/evidence/m3/01-fmt-validate-verify.txt`.

Part F — Reading the state

Three fields at the top of `terraform.tfstate` are worth separating, because they are easy to conflate:

```
{
  "version": 4,
  "terraform_version": "1.17.0",
  "serial": 15,
  "lineage": "6cdb4232-276c-7d22-9831-b53af1cd1da1",
  ...
}
```

`serial` is a version counter for the state file itself.

It starts at 1 and increases by one every time Terraform writes the file — each apply, each refresh that finds a change, each `state mv`. It has nothing to do with any resource's ARN or ID. Its job is to let a remote backend detect a stale write: if my local copy says serial 14 and the backend already holds 15, somebody else applied in between and my write is rejected.

`lineage` is a UUID for the state's whole history.

It is generated once, when the state is first created, and never changes however many applies follow. Two state files with different lineages are unrelated histories, even if they describe the same infrastructure — which is how Terraform catches somebody pointing at the wrong backend or restoring the wrong backup.

The resource's identity lives further down, per resource.

Each resource block in the state carries the address Terraform uses in the code (`aws_s3_bucket.artifacts`) and the attributes AWS returned, including `id` and `arn`. The data source shows the shape clearly:

```
        {
          "mode": "data",
          "type": "aws_caller_identity",
          "name": "current",
          "instances": [
            {
              "attributes": {
                "account_id": "4831*******",
                "arn": "arn:aws:iam::4831*******:user/adminUser",
                "id": "4831*******"
              }
            }
          ]
        }
```

So the link the assignment asks about is **address to ID**, not serial to ARN. The address is the name my code uses; the ID is the name AWS uses; the ARN is simply one more attribute AWS returned alongside it. State is the lookup table between the two, and that table is the entire reason a second apply is a no-op instead of a second bucket — without it Terraform could not tell that the resource in my file is the bucket that already exists in the account.

Safety note: the bucket is left standing. Session 3 begins by destroying it, once the code has been rewritten as a module, and `-auto-approve` stays out of every apply run on a laptop.